Smart Wall Paint Visualizer - Project Report

Oct 14, 2026 · @akshad

1. Overview

Smart Wall Paint Visualizer is a MEAN-stack web application that lets a person preview paint colours and wall patterns on a photo of their own room before buying paint. It was built as the Full-stack Web Development internship project assigned through Unified Mentor.

Problem. Customers choose wall colours from small shade cards or showroom displays, and these do not show how a colour looks in their own room's light and space. Wrong choices lead to dissatisfaction and repainting costs.

Objectives

- Let users preview paint on their actual room photos with a realistic, interactive result.
- Reduce incorrect paint selection and repainting expense, and raise confidence in interior decisions.
- Support designers, painters and retailers during consultations.
- Demonstrate a complete, secure, end-to-end MEAN stack implementation.

Source code: github.com/erakshadali/smart-wall-paint-visualizer

2. Features implemented

Every in-scope item of the project brief is built. Items listed as out of scope for Phase 1 (automatic AI wall detection, AR camera preview, a mobile app, payments) were deliberately left out.

User features

- Registration and login with JWT.
- Upload a JPG or PNG room photo, after confirming the photo is original and owned by the user.
- Manual wall selection with a **polygon** tool and a **brush** tool; points can be dragged, walls deleted, and every step undone or redone.
- Solid paint colours from a 40-colour palette with search and favourites, plus six tiled wall patterns with an adjustable pattern size.
- Opacity, brightness and a matte, satin or glossy finish for each wall.
- Dual-tone rooms: each wall holds its own colour or pattern.

- Before and after slider, saved designs, duplicate a design to try another scheme, and PNG download.
- A "How to use" guide inside the app and a lighting disclaimer on every preview.

Admin features

- Add, edit and delete colours and patterns, including pattern image upload.
- View all users, saved designs and per-user design counts; remove designs.
- A KPI dashboard that refreshes every 10 seconds, with a 14-day uploads chart and a table view.
- A settings screen for system configuration values.

Pages (7 routes, all connected; the admin page has six tabs)

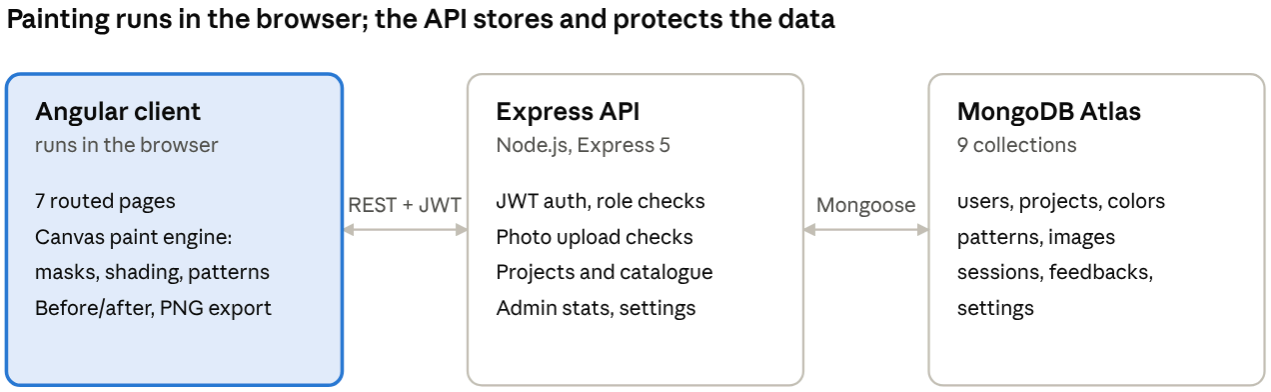
Page	Purpose
Home	Landing page and entry points
Login / Register	Authentication
Upload	Photo upload with ownership confirmation
Editor	Wall selection, colour and pattern preview, compare
My designs	Saved designs, duplicate, delete, feedback
How to use	User guide
Admin: Dashboard	KPIs and chart
Admin: Colours, Patterns, Users, Designs, Settings	Content and system management

3. Technology stack and architecture

The project uses the MEAN stack the brief requires, with TypeScript on both sides. The browser does the image processing, and the API stores and protects the data.

Layer	Technology	Role
Frontend	Angular 20, TypeScript, SCSS, HTML5 Canvas	Pages, wall editor, paint engine
Backend	Node.js, Express 5, REST APIs, JWT	Authentication, uploads, designs, admin

Layer	Technology	Role
Database	MongoDB with Mongoose	Users, designs, colours, patterns, images, analytics data
Hosting	Vercel (client and API), MongoDB Atlas	Client, API and database (live)



system architecture · 3 parts

The browser calls the Express API over REST with a JWT token, and the API reads and writes MongoDB through Mongoose.

4. Database design

Data lives in nine MongoDB collections, defined with Mongoose schemas and matching the data requirements in the brief (users, projects, colours, patterns, admin data).

Collection	Main fields
users	name, email (unique), password hash, role (user or admin), favourite colours and patterns, created date
projects	owner, title, original image URL, result image URL, walls, created and updated dates
projects.walls	type (polygon or brush), points (x, y), brush size, colour, pattern, pattern scale, brightness, opacity, finish
colors	name, hex code, brand, finishes, category tags such as Living Room or Accent
patterns	name, description, style, image URL, scale

Collection	Main fields
images	name, content type, binary data (room photos and pattern tiles)
sessions	user, seconds (average session duration KPI)
feedbacks	user, rating 1 to 5, comment (satisfaction KPI)
settings	key, value (admin-managed system configuration)

Uploaded photos are stored in MongoDB itself, which the brief allows ("in the database or a dedicated media store"). This keeps the API stateless, so it also works on hosts whose disk is erased on every deploy.

5. How the realistic paint works

A flat colour fill looks fake, so the editor keeps the photo's own light and shadow. The logic is in one file, `paint-engine.ts`, and runs entirely in the browser on an HTML5 canvas.

1. The photo is read once and the brightness of every pixel is stored.
2. Each wall is turned into a mask from its polygon or brush stroke, with soft edges.
3. The average brightness of the wall is measured.
4. For every pixel in the mask, its brightness relative to that average is applied to the chosen colour, so shadows stay dark and bright areas stay bright.
5. The finish changes how strongly that variation is kept (matte softer, glossy stronger with a highlight), and the brightness and opacity sliders adjust the result.
6. For patterns, the tile is repeated across the wall and its pixels are used in place of the single colour, with the same shading applied.

The before and after slider simply draws the untouched photo over the left part of the result.

6. Security and data handling

The brief asks for secure handling of images and user data. The following measures are in place:

- Passwords are hashed with bcrypt, and sessions use signed JWT tokens.
- Admin routes check the user's role, and a user can open, change or delete only their own designs.
- Uploads accept only JPG and PNG up to 8 MB. The server checks the real file signature, not just the type the browser reports, and stores files under random names.

- Users must confirm each photo is original and owned by them, as the brief requires.
- The API uses Helmet headers, a CORS allow-list, rate limiting and request size limits.
- Secrets such as the database password and JWT secret stay in an ignored `.env` file and are never committed.
- Every preview carries a notice that results vary with lighting, screen calibration and wall texture.

7. Testing and results

The application was tested manually against a live MongoDB Atlas database. There is no automated test suite yet, which is listed under future work.

Area	What was checked	Result
API	Health check, colour list, admin login, user blocked from admin routes	Passed
Upload rules	Missing ownership confirmation rejected; fake file renamed to .png rejected; valid photo accepted	Passed
Designs	Walls saved and reloaded; a wall with too few points rejected; delete returns 204	Passed
Admin KPIs	Counts of images, designs, users and session time update after use	Passed
Editor in browser	Polygon outlining, colour and pattern fill, dual-tone walls, before and after slider, save, duplicate	Passed
Admin in browser	Dashboard, chart, settings screen	Passed
Layout	Desktop and tablet width checked; the phone top bar was fixed after it overflowed	Passed
Builds	Server type check and Angular production build	Passed

The brush tool, dragging wall points, and undo and redo were built and compile, but were not clicked through in the final manual run.

8. Challenges and solutions

Challenge	Solution
A flat colour looked unrealistic	Kept each pixel's brightness relative to the wall average, so shadows and light stay visible
Hosts like Render erase local files on every deploy, which would lose uploaded photos	Moved image storage into MongoDB and serve images through the API
Re-running the seed script created new colour IDs, so saved designs lost their colour link	Seed now updates colours by name, keeping IDs stable
Duplicated designs share one photo, so deleting one design could remove the other's image	Delete removes the photo only when no other design uses it
Counting duplicates as new uploads inflated the images-uploaded KPI	The KPI counts distinct photos
The database password was saved on the wrong line of the environment file, so the first connection failed	Found the misplaced line, moved it into the connection string and re-ran the seed
Phone screens cut off the editor's top bar	Added a small-screen layout so the buttons wrap

9. Deployment, links and how to run

Item	Link or status
Source code	GitHub repository
Live application	smart-wall-paint-visualizer-zeta.vercel.app (client and API both on Vercel)
Database	MongoDB Atlas free cluster

The repository README holds the full setup steps, environment variables, API reference and deployment guide. In short:

1. Start the API: `cd server`, install packages, copy `.env.example` to `.env`, set `MONGODB_URI` and `JWT_SECRET`, run `npm run seed`, then `npm run dev`.

2. Start the client: `cd client`, run `npm install`, then `npm start`, and open `http://localhost:4200`.
3. The app is deployed on Vercel as two projects: the Angular client, and the Express API running as a serverless function. The README has the full deployment guide.

10. Learnings and future enhancements

What I learned

- Building a complete MEAN-stack product end to end: Angular standalone components and signals, an Express and TypeScript API, and MongoDB modelling.
- Working with the HTML5 canvas: pixel-level image processing, masks, hit testing, undo and redo history, and keeping rendering fast.
- Designing for real deployment early, which is why image storage moved into the database.
- Writing secure upload handling and role-based access.
- Checking my work against the written requirements and closing the gaps it showed.

Future enhancements (from the brief)

- Automatic AI-based wall detection.
- Advanced textures and 3D previews.
- Day and night lighting simulation.
- Integration with paint brands and retailers.

Technical next steps

- Automated unit and end-to-end tests.
- Finer admin permissions beyond the two current roles.